Introduction

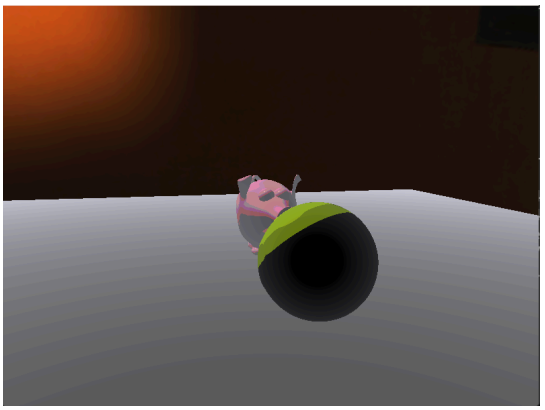
The first coursework of COMP3015 Games Graphics Pipeline asked to create a project utilizing a custom shading model showcasing a texture based technique.

The current scene of the project contains two objects. One that was given from a lab and an other I found online.
The project also contains a skybox, fog, toon shading , gramma correction.

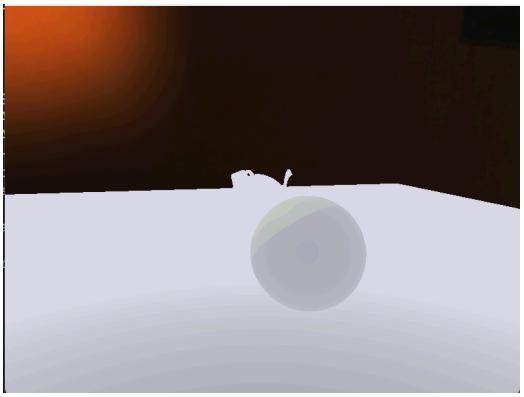
Pig was given in Lab2.
Eye is from https://free3d.com/3d-model/eyeball--33237.html?dd_referrer=

Link to git = https://github.com/sniffleslou-lab/CW1GrapphicsPipeline

Scene Overview



Scene with fog weakened



Scene with fog set to be more visible.

```cpp
void SceneBasic_Uniform::initScene()
{
    compile();
    glEnable(GL_DEPTH_TEST);


    //texture
    int width, height, channels;
    unsigned char* data = stbi_load("media/brat-album-font.png", &width, &height, &channels, 4);
    glGenTextures(1, &eyeballTex);
    glBindTexture(GL_TEXTURE_2D, eyeballTex);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,
                 GL_RGBA, GL_UNSIGNED_BYTE, data);
    glGenerateMipmap(GL_TEXTURE_2D);
    //textures
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    //fog
    prog.setUniform("FogColor", vec3(0.7f, 0.7f, 0.8f)); // should be greylish blue
    prog.setUniform("FogStart", 2.0f);//0.5 for strong | weaker = 2
    prog.setUniform("FogEnd", 12.0f);// 3.0 for strong | weaker = 12


    //cube
    projection = mat4(1.0f);
    angle = glm::radians(90.0f);
    cubeTex = Texture::loadHdrCubeMap("media/pisa-hdr/pisa");

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);//binding
    //skybox
    sky = new SkyBox(50.0f);



    //set the camera and the 3 lights
    view = glm::lookAt(vec3(3.6f, 0.75f, 0.75f),
        vec3(0.0f, 0.1f, 0.0f),
        vec3(0.0f, 1.0f, 0.0f));
    projection = mat4(1.0f);
    // Single light for per-fragment Phong
    vec3 lightPosEye = vec3(view * vec4(-2.0f, 4.0f, 2.0f, 1.0f));
    prog.setUniform("LightPosition", lightPosEye);
    prog.setUniform("LightLa", vec3(0.2f));
    prog.setUniform("LightLd", vec3(0.8f));
    prog.setUniform("LightLs", vec3(0.8f));


}
```

On the image of the right, you can I integrated texturing, fog, cube and the camera with phong lighting.

Fragment Shaders

```cpp
///for textures
uniform int UseTexture;
uniform sampler2D Tex;
in vec2 TexCoord;

//fog
uniform vec3 FogColor;
uniform float FogStart;
uniform float FogEnd;

vec3 phongModel(vec3 position, vec3 n)
{
    vec3 s = normalize(LightPosition - position);
    vec3 v = normalize(-position);
    vec3 r = reflect(-s, n);

    // Ambient
    vec3 ambient = LightLa * Ka;

    // Diffuse
    float sDotN = max(dot(s, n), 0.0);
    vec3 diffuse = LightLd * Kd * sDotN;

    // Specular
    vec3 specular = vec3(0.0);
    if (sDotN > 0.0) {
        specular = LightLs * Ks * pow(max(dot(r, v), 0.0), Shininess);
    }

    return ambient + diffuse + specular;
}

void main()
{
    vec3 n = normalize(Normal);
    vec3 texColor = texture(Tex, TexCoord).rgb;

    vec3 lighting = phongModel(Position, n);

    vec3 baseColor = (UseTexture == 1) ? texColor : Kd;

    //toon shading
    float levels = 4.0;
    vec3 toonLighting = floor(lighting * levels) / levels;
    vec3 litColor = baseColor * toonLighting;

    //fog
    float dist = length(Position);
    float fogFactor = clamp((dist - FogStart) / (FogEnd - FogStart), 0.0, 1.0);

    //FragColor = vec4(baseColor * lighting, 1.0);

    vec3 finalColor = mix(litColor, FogColor, fogFactor);
    FragColor = vec4(pow(finalColor, vec3(1.0/2.2)), 1.0); //gamma
}
```

On the left is my fragment shader. I took the fragment shader I used from my lab 3 work and adjusted it to include additional techniques such as toon shading.

PhongModel
Ambient simulats the indirect light in the scene.
Diffuse depends on angel that the light is directed at and the surface, giving objects its shading.

Specular heights shiny reflections. It contons how sharp or soft the highlight is.

Toon shading is the line that gets the levels, lighting and lit color which calculates the lighting set and the base colour to give the objects a cartoonish look.

Fog is we the get values from FogStart and fogEnd, when we get distence from fragment to camera. In the end we blend the lit colour with the foh colour in vec3 final colour.

```
//fog
prog.setUniform("FogColor", vec3(0.7f, 0.7f, 0.8f)); // should be greylish blue
prog.setUniform("FogStart", 2.0f);//0.5 for strong | weaker = 2
prog.setUniform("FogEnd", 12.0f);// 3.0 for strong | weaker = 12
```

Basic texture sampling

```
//texture
int width, height, channels;
unsigned char* data = stbi_load("media/brat-album-font.png", &width, &height, &channels, 4);
glGenTextures(1, &eyeballTex);
glBindTexture(GL_TEXTURE_2D, eyeballTex);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,
             GL_RGBA, GL_UNSIGNED_BYTE, data);
glGenerateMipmap(GL_TEXTURE_2D);
```

```
prog.setUniform("UseTexture", 0);
prog.setUniform("Kd", 1.0f, 0.6f, 0.7f);
prog.setUniform("Ks", 1.0f, 1.0f, 1.0f);
prog.setUniform("Ka", 0.6f, 0.3f, 0.4f);
prog.setUniform("Shininess", 200.0f);


model = mat4(1.0f);
model = glm::rotate(model, glm::radians(90.0f), vec3(0.0f, 1.0f, 0.0f))
setMatrices();
mesh->render();

//for eye
prog.setUniform("UseTexture", 1);
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, eyeballTex);
prog.setUniform("Tex", 0);

///material for eyeball

prog.setUniform("Kd", 1.0f, 1.0f, 1.0f);
prog.setUniform("Ks", 0.2f, 0.2f, 0.2f);
prog.setUniform("Ka", 0.3f, 0.3f, 0.3f);
prog.setUniform("Shininess", 50.0f);
model = mat4(1.0f);
model = glm::translate(model, vec3(1.5f, 0.0f, 0.0f));
model = glm::scale(model, vec3(0.2f));
setMatrices();
mesh2->render();


prog.setUniform("Kd", 0.4f, 0.4f, 0.4f);
prog.setUniform("Ks", 0.9f, 0.9f, 0.9f);
prog.setUniform("Ka", 0.5f, 0.5f, 0.5f);
prog.setUniform("Shininess", 180.0f);
prog.setUniform("UseTexture", 0);


model = mat4(1.0f);
model = glm::translate(model, vec3(0.0f, -0.45f, 0.0f));
setMatrices();
plane.render();
```

For the texturing I used the lap sample using stb_image in my header, and used glGenTexture to texture object, when applied the data of the texture using glTextImage2d.

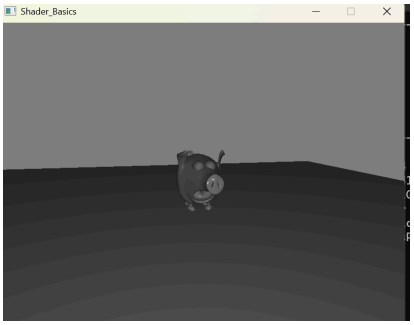
I bounded the image before rendering the object.

To control what has the texture applied, I used the line vec3 baseColor = (UseTexture == 1). So basically textures that have 1 will have the image applied to it, while images that have 0 will not have the image applied to it.

I used the brat album cover as the image to be applied to the eye object since I had it laying around on my pc.

Skybox

The image that has grey showcases the lack of a skybox, using it as a example on how it looks. Without applying anything to it.



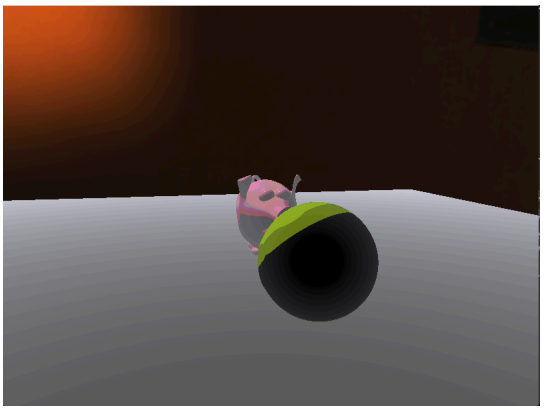
Example without skybox

```
//cube
projection = mat4(1.0f);
angle = glm::radians(90.0f);
cubeTex = Texture::loadHdrCubeMap("media/pisa-hdr/pisa");

glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);//binding
//skybox
sky = new SkyBox(50.0f);
```

I used the texture.cpp code that was given in Lab4, and the skymap was that given (as I couldn't find suitable one online).
Using the code I load the file and give the item name.

```
void SceneBasic_Uniform::compile()
{
    std::cout << "Working directory: " << std::filesystem::current_path() << std::endl;
    try {


        prog.compileShader("shader/PerFragment_phong.vert");
        prog.compileShader("shader/PerFragment_phong.frag");

        prog.link();
        prog.use();

        skyProg.compileShader("shader/skybox.vert");
        skyProg.compileShader("shader/skybox.frag");
        skyProg.link();

    } catch (GLSLProgramException &e) {
        cerr << e.what() << endl;
        exit(EXIT_FAILURE);
    }
}

void SceneBasic_Uniform::update( float t )
{
    //update your angle here
  //; if (m_animate) {
    //    angle += 0.1f;
    ; //  if (angle >= 360.0f) angle -= 360.0f;
    //}
    float deltaT = t - tPrev;
    if (tPrev == 0.0f) deltaT = 0.0F;
    tPrev = t;

    angle += rotSpeed * deltaT;
    if (angle > glm::two_pi<float>())
        angle -= glm::two_pi<float>();


}

void SceneBasic_Uniform::render()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);


    //skybox
    glDepthMask(GL_FALSE);
    skyProg.use();

    mat4 viewNoTrans = mat4(mat3(view));
    mat4 MVP = projection * viewNoTrans;

    skyProg.setUniform("MVP", MVP);
    skyProg.setUniform("SkyBoxTex", 0);
    sky->render();
    glDepthMask(GL_TRUE);

    prog.use();
```

I created a shader skybox.vert and skybox.frag, and I compile both.
Calling them again before I render objects in render.



Example with skybox

Image processing

Gamma correction

```
FragColor = vec4(pow(finalColor, vec3(1.0/2.2)), 1.0); //gamma
```

Allows the colours in my scene display correctly on a standard monitor. Without the line the scene would appear darker.